Model Comparison

Models

All models include temporal weighting to ensure more recent fixtures have a larger effect on the posterior. The models are:

- `PoissonModel`
    - Tries and penalties modelled separetly as Poisson processes. Poisson lambda is calculated at exponential of a linear combination of:
        - attack parameter of team minus fefense parameter of opposition (learned at team level)
            - Zero-sum normmal prior on attack and defense parameters, with half normal hyper-parameter for the standard devation of these distributions.
        - Home-team advantage
            - Normal prior
        - interecpt -Normal prior with mean set to the log mean number of tries/penatlies in the dataset.
    - Conversions are modelled as Binomial distribution with a beta prior at the team level, with mu set to the average conversion rate.
- `PoissonModel_DG`
    - As `PoissonModel` but also models drop goals as Zero-inflated Poisson (with psi parameter (on logit scale) modelling probability of being range of drop-goal and lambda modelled with attacking parameter, intercept and home advantage). Not too familiar with this so a bit experimental.
- `PoissonModel_latAtt`
    - As `PoissonModel` but with a latent attack parameter (normal prior) at the game level, accountring for more open higher scoring games.
- `NegBinModel`
    - As `PoissonModel` but tries and penlties modelled as negative binomial ot account for potential overdispersion. Exponential prior on overdispersion parameter.
- `KISS`
    - Keep it simple stupid. Modelled score on its own as negative binomial (att, def, home and int parameters used for lambda and overdispersion modeeled as in `NegBinModel`). Note that this cannot be used for tournament siumulation as bonus points cannot be modelled without try count.
- `KISSS`

- Keep is simpler stupid. As `KISS` but with a single strenght parameter per team combining defense and attack parameters, and no home advantage.

Note that here I test modells trained up to a certain date on all future fixtures of focal teams. Ideally the model would be refit on each result up to the fixture being predicted but this would be too slow here. Therefore tests are simulating each fixture based on a teams predicted form at the cutoff.

Setup

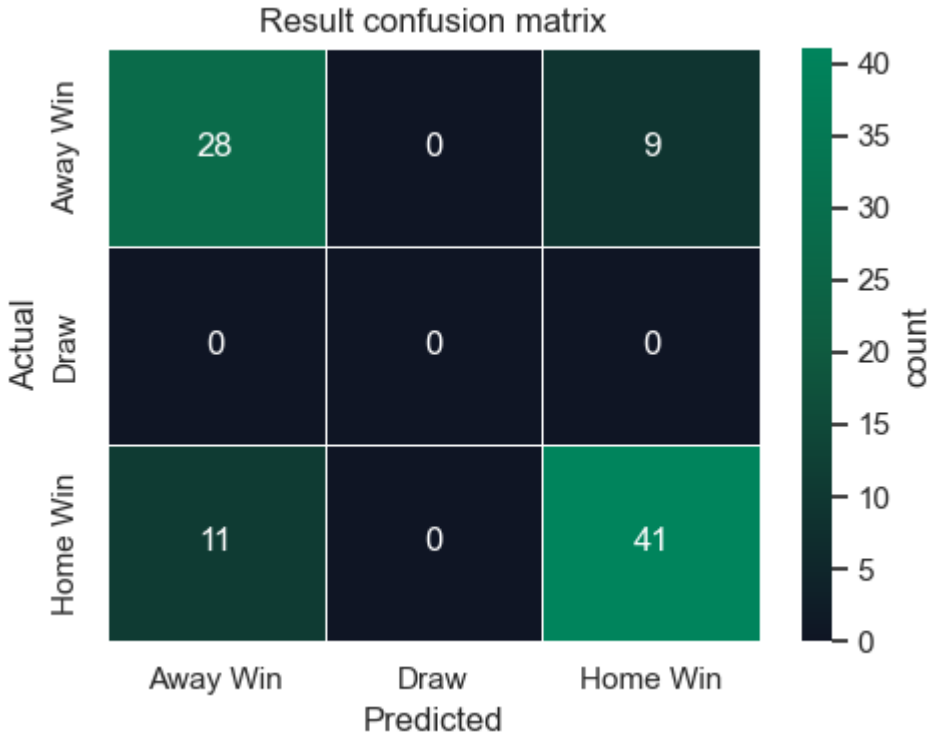
In [1]:
```
from rugbyModels import *
np.random.seed(1)
import warnings
warnings.filterwarnings("ignore")

data= pd.read_csv("final_data_1.csv")
data["date"] = pd.to_datetime(data["date"])

dataDGs = data.copy() #save this for the model with separate drop goals

#think its reasoable to combine these for models without dgs - shouldnt overwrit
data["home_penalties"] = data["home_penalties"]+data["home_drop_goals"]
data["away_penalties"] = data["away_penalties"]+data["away_drop_goals"]

#train-test split
cutoff = "2025-01-01"
dfTest =  data[data["date"] >= cutoff]
dfTrain = data[data["date"] < cutoff]

dataDGsTest = dataDGs[dataDGs["date"] >= cutoff]
dataDGsTrain = dataDGs[dataDGs["date"] < cutoff]
```

PoissonModel

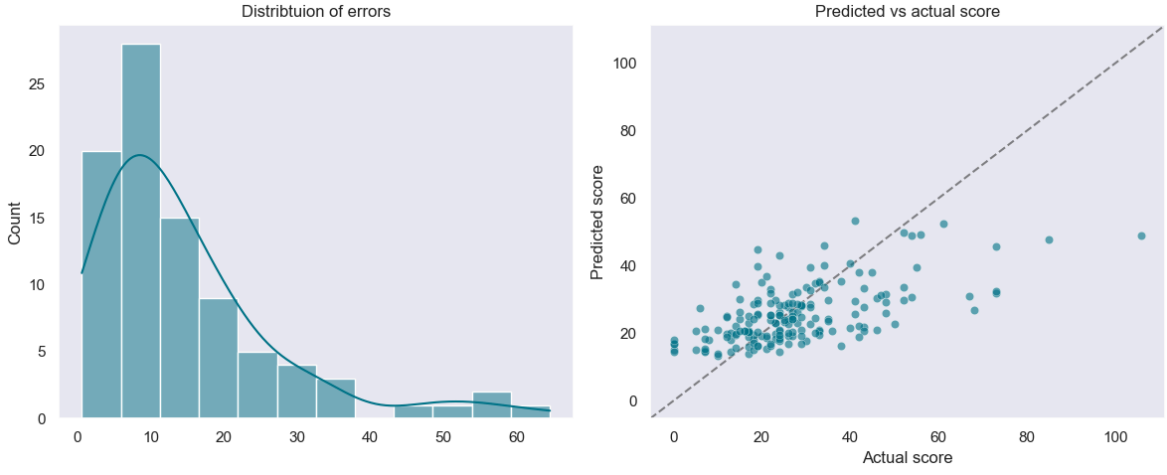
In [2]:
```
poisson = PoissonModel()
poissonFit = poisson.fit(dfTrain)
_ = poissonFit.eval_result(dfTest)
_ = poissonFit.eval_score(dfTest)
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [home, sd_att, sd_def, att, def_, int_try, sd_dis_clean, sd_dis_force, dis_
clean, dis_force, int_pen, p_conv]
Output()
```

```
Sampling 4 chains for 1_500 tune and 2_500 draw iterations (6_000 + 10_000 draws
total) took 409 seconds.
Chain 3 reached the maximum tree depth. Increase `max_treedepth`, increase `targe
t_accept` or reparameterize.
The rhat statistic is larger than 1.01 for some parameters. This indicates proble
ms during sampling. See https://arxiv.org/abs/1903.08008 for details
The effective sample size per chain is smaller than 100 for some parameters.  A h
igher number is needed for reliable rhat and ess computation. See https://arxiv.o
rg/abs/1903.08008 for details
```




```
Match result accuracy = 0.7752808988764045


Score level evaulation:

    Number of usable tests: 89

    Average observed score = 27.617977528089888 per team per game.
    Average predicted score = 27.360608426966294 per team per game.
    MAE score = 13.529046067415731 per team per game.
    MAE score difference = 14.640684269662923
```



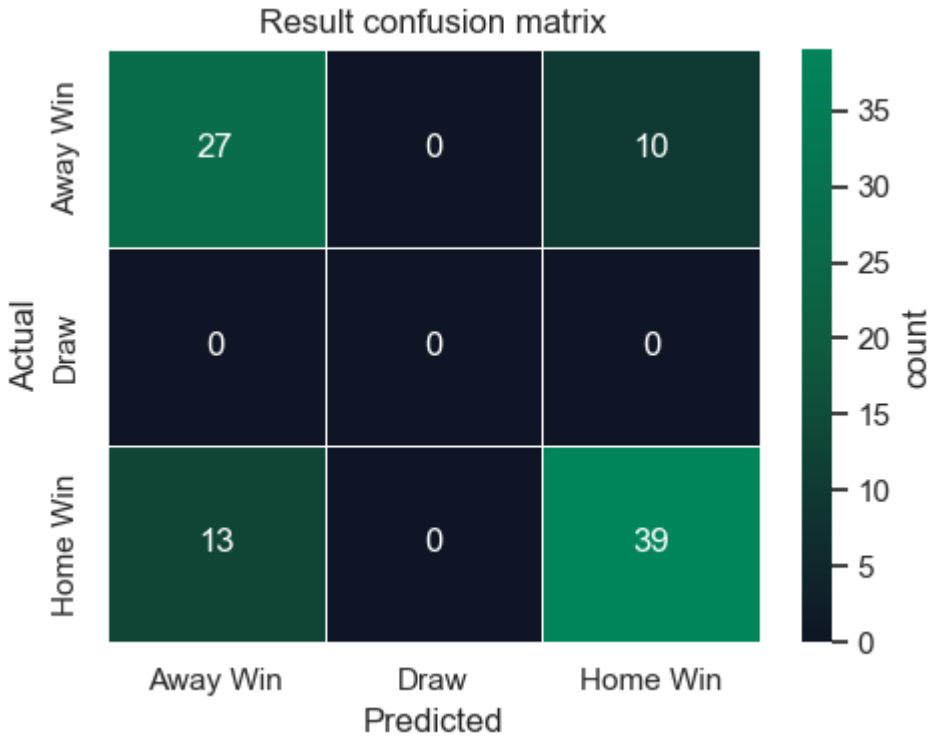
NegBinModel

Not unreasonable that try and penalty counts would be overdispersed.

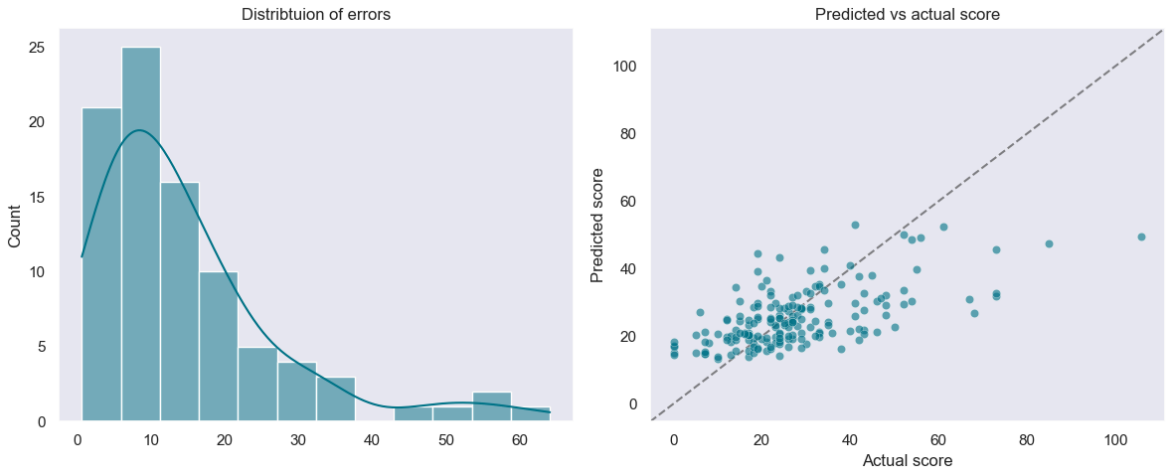
In [3]:
```
nb = NegBinModel()
nbFit = nb.fit(dfTrain)
nbFit.eval_result(dfTest)
poissonFit.eval_score(dfTest)
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [home, sd_att, sd_def, att, def_, int_try, alpha, sd_dis_clean, sd_dis_forc
e, alphaPen, dis_clean, dis_force, int_pen, p_conv]
Output()
```

```
Sampling 4 chains for 1_500 tune and 2_500 draw iterations (6_000 + 10_000 draws
total) took 531 seconds.
There were 2 divergences after tuning. Increase `target_accept` or reparameteriz
e.
The rhat statistic is larger than 1.01 for some parameters. This indicates proble
ms during sampling. See https://arxiv.org/abs/1903.08008 for details
The effective sample size per chain is smaller than 100 for some parameters.  A h
igher number is needed for reliable rhat and ess computation. See https://arxiv.o
rg/abs/1903.08008 for details
```




```
Match result accuracy = 0.7415730337078652


Score level evaulation:

    Number of usable tests: 89

    Average observed score = 27.617977528089888 per team per game.
    Average predicted score = 27.33551235955057 per team per game.
    MAE score = 13.51118202247191 per team per game.
    MAE score difference = 14.601546067415732
```



Out[3]: 14.601546067415732

Seems to be slightly worse.

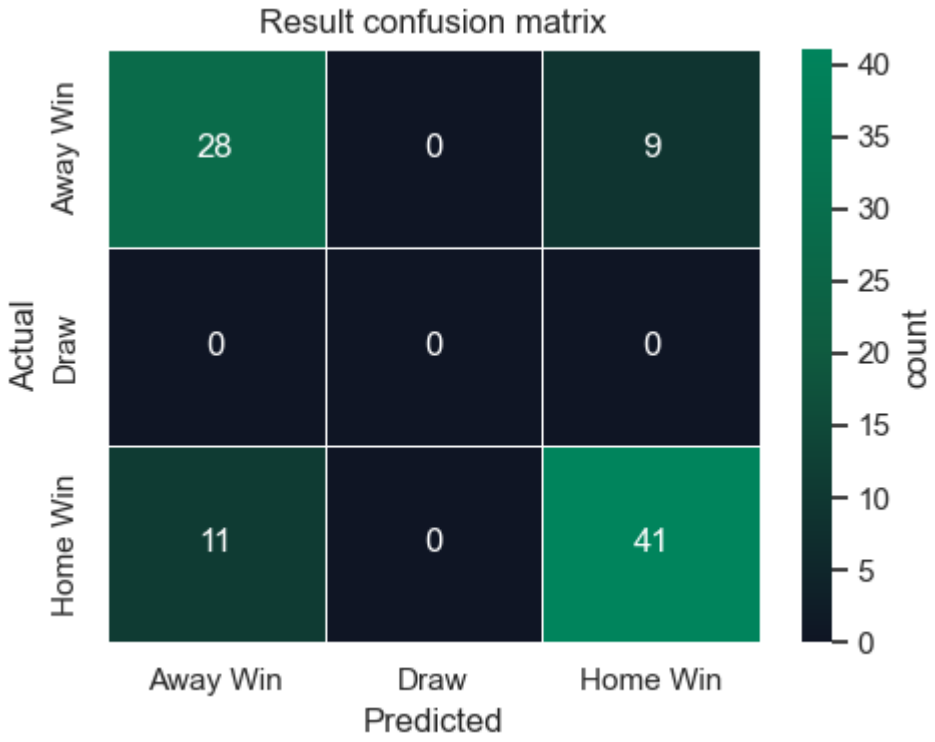
PoissonModel_latAtt

Can a match openness parameter imporve score predictions?

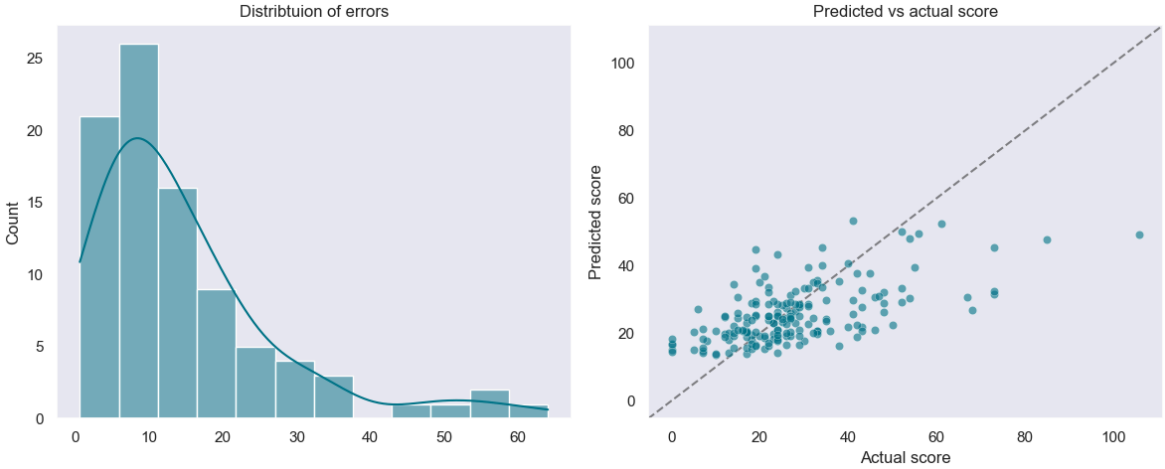
In [4]:
```
poissonLA = PoissonModel_latAtt()
poissonLAFit = poissonLA.fit(dfTrain)
_ = poissonLAFit.eval_result(dfTest)
_ = poissonLAFit.eval_score(dfTest)
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [home, sd_att, sd_def, att, def_, int_try, sigma_game, game_effect, sd_dis_
clean, sd_dis_force, dis_clean, dis_force, int_pen, p_conv]
Output()
```

```
Sampling 4 chains for 1_500 tune and 2_500 draw iterations (6_000 + 10_000 draws
total) took 148 seconds.
The rhat statistic is larger than 1.01 for some parameters. This indicates proble
ms during sampling. See https://arxiv.org/abs/1903.08008 for details
The effective sample size per chain is smaller than 100 for some parameters.  A h
igher number is needed for reliable rhat and ess computation. See https://arxiv.o
rg/abs/1903.08008 for details
```




```
Match result accuracy = 0.7752808988764045


Score level evaulation:

    Number of usable tests: 89

    Average observed score = 27.617977528089888 per team per game.
    Average predicted score = 27.34460898876404 per team per game.
    MAE score = 13.507717977528092 per team per game.
    MAE score difference = 14.622647191011236
```



PoissonModel_DG

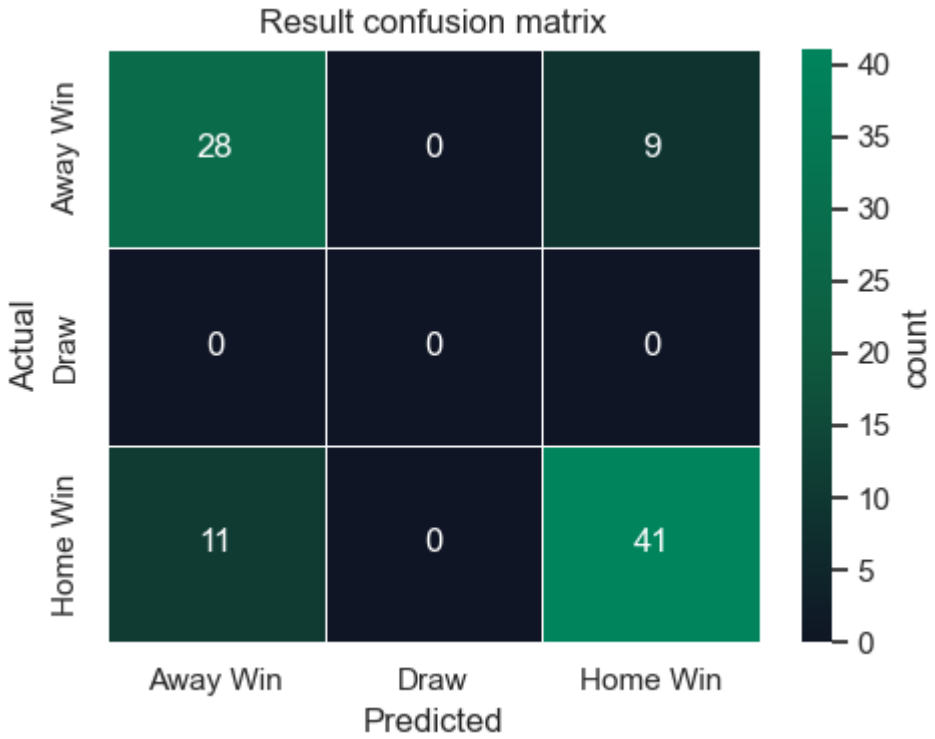
What if we add drop goals?

In [5]:
```
poissonDG = PoissonModel_DG()
poissonDGFit = poissonDG.fit(dfTrain)
```

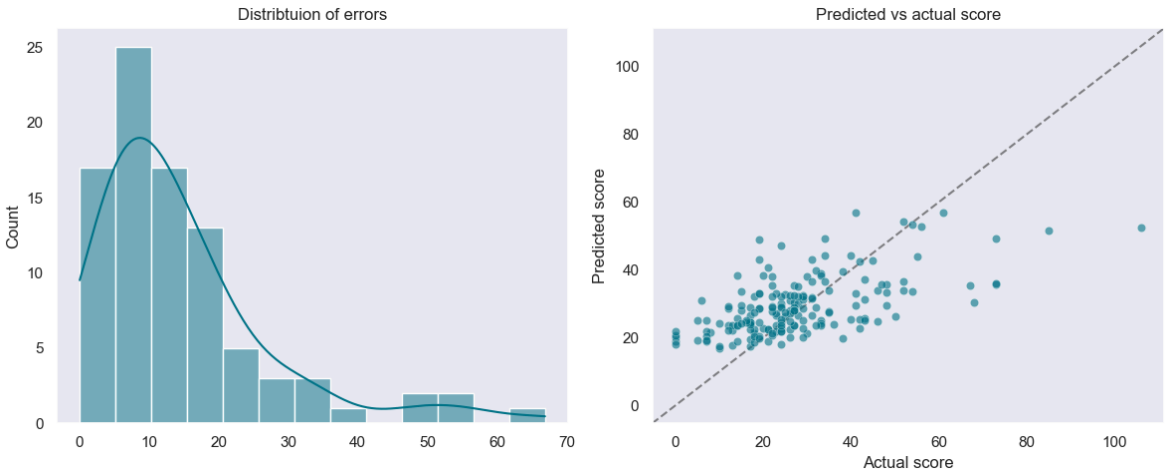
```
_ = poissonDGFit.eval_result(dfTest)
_ = poissonDGFit.eval_score(dfTest)
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [home, sd_att, sd_def, att, def_, int_try, sd_dis_clean, sd_dis_force, dis_
clean, dis_force, int_pen, p_conv, sd_att_dg, att_dg, int_dg, psi_logit_dg]
Output()
```

```
Sampling 4 chains for 1_500 tune and 2_500 draw iterations (6_000 + 10_000 draws
total) took 385 seconds.
Chain 1 reached the maximum tree depth. Increase `max_treedepth`, increase `targe
t_accept` or reparameterize.
The rhat statistic is larger than 1.01 for some parameters. This indicates proble
ms during sampling. See https://arxiv.org/abs/1903.08008 for details
The effective sample size per chain is smaller than 100 for some parameters.  A h
igher number is needed for reliable rhat and ess computation. See https://arxiv.o
rg/abs/1903.08008 for details
```




```
Match result accuracy = 0.7752808988764045


Score level evaulation:

    Number of usable tests: 89

    Average observed score = 27.617977528089888 per team per game.
    Average predicted score = 29.29770112359551 per team per game.
    MAE score = 14.127411797752814 per team per game.
    MAE score difference = 14.648295505617977
```



Not much better, and we dont need DGs to simulate the tournement.

KISS

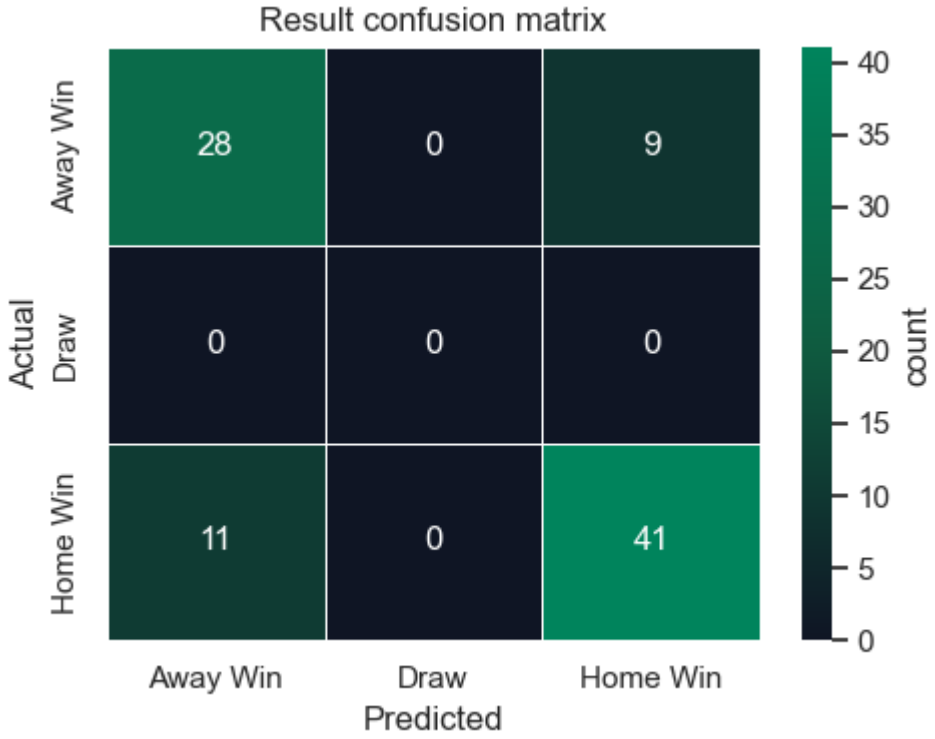
Out of interest, is a simpler model more accurate?

In [6]:
```
kiss = KISSModel()
kissFit = kiss.fit(dfTrain)
_ = kissFit.eval_result(dfTest)
_ = kissFit.eval_score(dfTest)
```

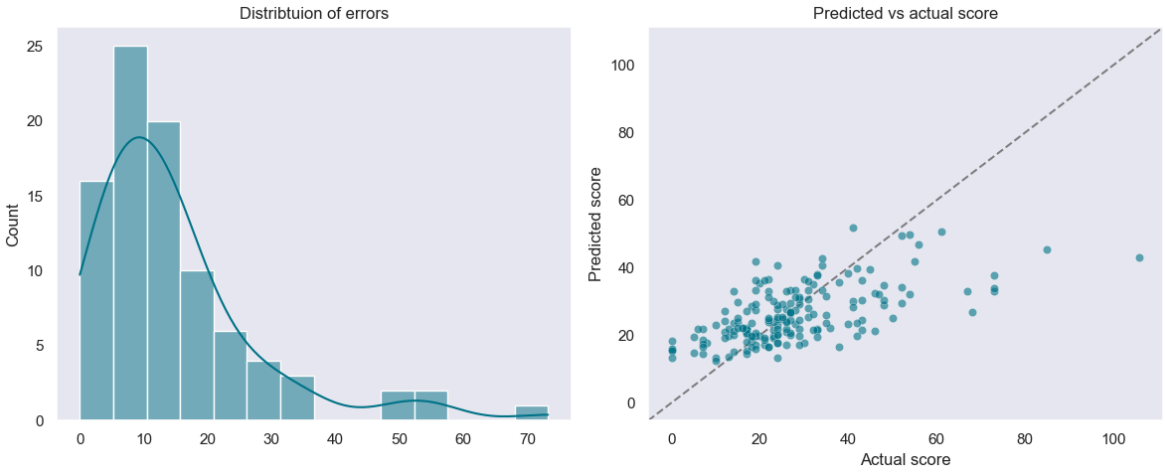
```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [home, sd_att, sd_def, att, def_, int, alpha]
```

Output()

```
Sampling 4 chains for 1_500 tune and 2_500 draw iterations (6_000 + 10_000 draws total) took 80 seconds.
```



```
Match result accuracy = 0.7752808988764045


Score level evaulation:

    Number of usable tests: 89

    Average observed score = 27.617977528089888 per team per game.
    Average predicted score = 27.882053370786522 per team per game.
    MAE score = 13.200079213483145 per team per game.
    MAE score difference = 14.706103370786515
```



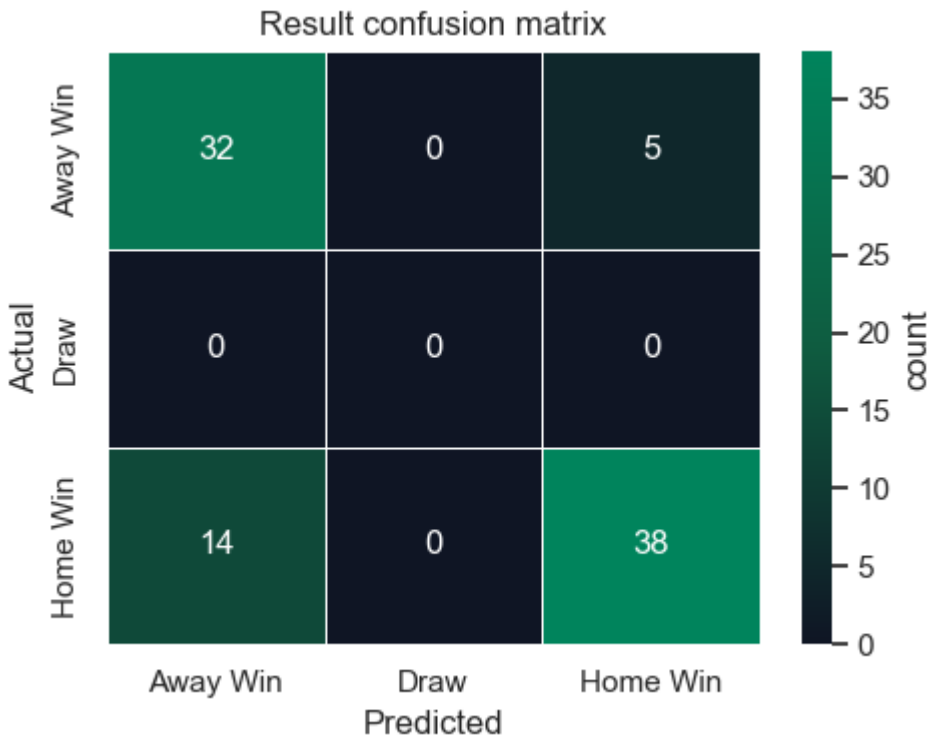
KISSS

The simpilest possible model.

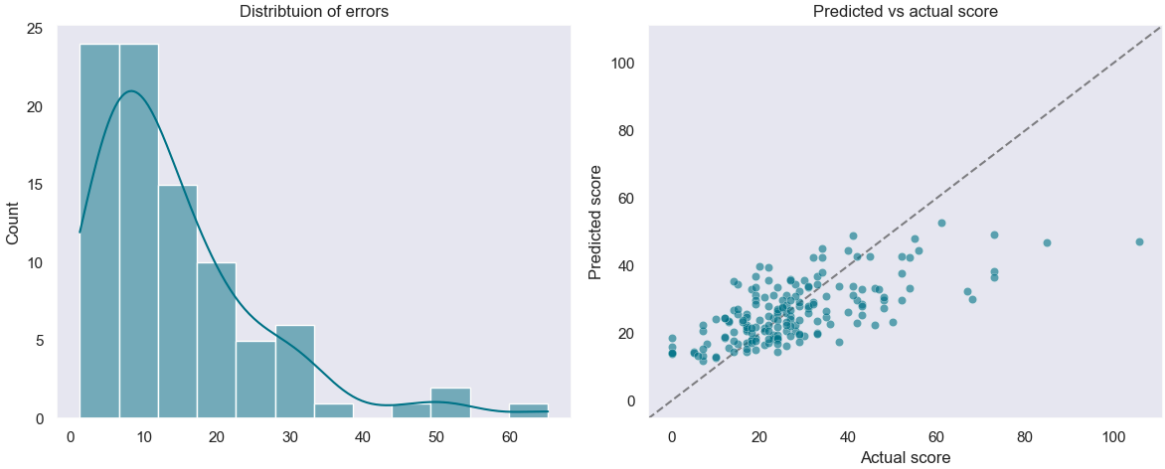
In [7]:
```
kisss = KISSSModel()
kisssFit = kisss.fit(dfTrain)
_ = kisssFit.eval_result(dfTest)
_ = kisssFit.eval_score(dfTest)
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [sd_strength, strength, int, alpha]
```
Output()

```
Sampling 4 chains for 1_500 tune and 2_500 draw iterations (6_000 + 10_000 draws total) took 58 seconds.
```




```
Match result accuracy = 0.7865168539325843


Score level evaulation:

    Number of usable tests: 89

    Average observed score = 27.617977528089888 per team per game.
    Average predicted score = 27.204507865168544 per team per game.
    MAE score = 12.893598876404496 per team per game.
    MAE score difference = 14.392975280898876
```



After all that, seems that the original model was best. The joys of machine learning.